

Recordatorio

En las iteraciones anteriores se implementó:

- **Iteración 01:** programa en consola (CLI) dentro de un único archivo, versión que sirvió entender la mecánica básica de la ruleta.
- **Iteración 02:** ejercicio de uso de ventanas para el módulo de login y registro en Swing, con usuarios hardcoded y almacenamiento en memoria.
- **Iteración 03:** aplicación del Principio de Responsabilidad Única (SRP), separando la lógica de la ruleta en clases.

1. Objetivos

Transformar la iteración 03 de la ruleta a un sistema que incorpore el patrón arquitectónico MVC, uso de tipo ENUM, encapsulamiento, constructores y getters/setters.

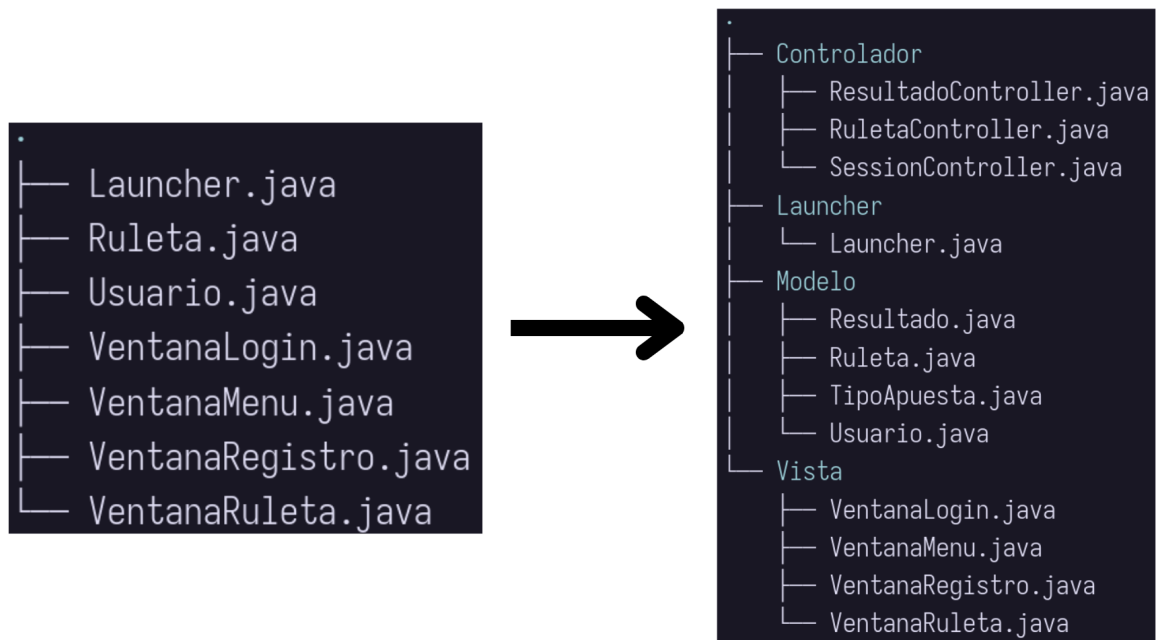


Figura 1: Referencia de iteración 03 a iteración 04

1.1 Objetivos específicos

- Encapsular todos los atributos de las clases **Usuario** y **Ruleta** declarando sus campos como **private**, de modo que no puedan ser accedidos directamente desde la vista.
- Implementar constructores con y sin parámetros en las clases principales:
 - **Usuario**: constructor con parámetros para inicializar **username**, **password** y **nombre**; constructor sin parámetros que cargue un usuario “invitado” por defecto.
 - **Ruleta**: constructor con parámetros que reciba el saldo inicial, y constructor sin parámetros que inicie con saldo 0.
- Usar getters y setters de manera explícita en las siguientes funcionalidades:
 - En la vista **Perfil**, el nombre del usuario debe mostrarse a través de un **getter** y actualizarse mediante un **setter** validado (evitar nombre vacío).
 - El saldo debe consultarse siempre con un **getter**, tanto en el menú principal como en la vista **Perfil**, sin acceder al atributo directamente.
 - La recarga de saldo debe implementarse como un método de negocio (**depositar(int monto)**) en lugar de un simple **setSaldo**, garantizando que solo se acepten montos válidos.
- Usar un **enum** para las apuestas:
 - Definir **TipoApuesta** { **ROJO**, **NEGRO**, **PAR**, **IMPAR** } en el modelo.
 - Modificar **Ruleta** para que sus métodos trabajen con este tipo.
 - En la vista, mapear el **JComboBox** al **enum**.
- Agregar una sección «Perfil» en el menú que muestre:
 - Nombre de usuario (editable con setter validado).
 - Usuario (no editable, solo getter).
 - Saldo actual (getter).
 - Botón para recargar saldo (usa método de negocio que modifica el saldo).
- Mostrar siempre el saldo actual en el menú principal a través de un **getter** del modelo.
- Reorganizar el código del sistema en un esquema MVC, asegurando que la Vista use exclusivamente getters, setters y constructores para interactuar con el Modelo a través del Controlador.

2. Ayudas y Sugerencias

2.1 Teoría

2.1.1. ¿Qué es MVC?

El Modelo–Vista–Controlador (MVC) es un patrón arquitectónico que surge de la necesidad de separar responsabilidades dentro de una aplicación, evitando que la lógica de negocio, la interfaz gráfica y el manejo de eventos se mezclen en una misma clase difícil de mantener.

- **Modelo:** Encapsula los datos y la lógica principal del sistema (por ejemplo, Usuario y Ruleta).
- **Vista:** Se encarga únicamente de mostrar información y capturar la interacción del usuario (ventanas en Swing).
- **Controlador:** Coordina la comunicación entre ambos, aplicando validaciones, reglas y decisiones antes de modificar el modelo o actualizar la vista.

2.1.2. ¿Por qué usar getters y setters?

El uso de getters y setters surge de la necesidad de aplicar el principio de **encapsulamiento**, el cual busca proteger los atributos internos de una clase, ya que si los valores pudiesen ser manipulados de manera directa, se corre el riesgo de perder control sobre su validez y consistencia, es decir, podrían asignarse datos inválidos (por ejemplo, un saldo negativo o un nombre vacío) sin que exista un mecanismo que lo impida.

- **Getters:** permiten consultar información de forma segura.
- **Setters:** controlan cómo y cuándo se modifica un atributo, pudiendo incorporar validaciones.

2.1.3. Constructores

Un «**constructor**» es un método especial de una clase que se ejecuta automáticamente al crear un objeto, cuya función principal es *inicializar sus atributos*, es decir, asignarles un valor de inicio a las instancias que se crean. Esto evita que los atributos queden vacíos, nulos o en un estado inconsistente.

Nota: Aunque no se escriba de manera explícita, Java siempre provee un constructor por defecto sin parámetros que permite instanciar la clase.

Existen dos tipos principales:

- **Constructor sin parámetros:** inicializa el objeto con valores por defecto, lo que resulta útil para pruebas rápidas o para instancias temporales.
- **Constructor con parámetros:** permite crear un objeto con valores definidos desde el inicio, asegurando que comience con un estado válido.

Además, en Java es posible la **sobrecarga de constructores**, lo que significa que una clase puede definir varios constructores con diferentes listas de parámetros, permitiendo crear objetos de distintas maneras según la necesidad.

Por ejemplo, en una clase `Usuario`, un constructor podría asignar un `id` de manera automática cada vez que se cree un nuevo objeto, garantizando unicidad:

```
public class Usuario {  
    private static int contador = 0;  
    private int id;  
    private String nombre;  
  
    // Constructor con parámetro  
    public Usuario(String nombre) {  
        this.id = ++contador; // asigna un id único automáticamente  
        this.nombre = nombre; // inicializa el atributo nombre  
    }  
  
    // Constructor sin parámetros (sobrecarga)  
    public Usuario() {  
        this("Invitado"); // delega al otro constructor  
    }  
}
```

De esta forma, los constructores no solo crean el objeto, sino que también establecen reglas iniciales y flexibilidad de instanciación que fortalecen la coherencia del sistema.

2.1.4. Modificadores de acceso

Los modificadores de acceso en Java definen el nivel de visibilidad que tendrán los atributos y métodos de una clase.

- **private:** restringe el acceso a la misma clase y se utiliza en atributos para evitar que sean manipulados de manera directa.

- **public:** permite que un elemento sea accesible desde cualquier otra clase, por lo que resulta apropiado para métodos que forman parte de la interfaz pública de la clase, como los getters y setters.
- **protected:** otorga acceso dentro de la misma clase, dentro del mismo paquete y también a las subclases, lo cual resulta útil en escenarios de herencia.

2.1.5. Buenas prácticas en la Vista

La «**Vista**» no debe contener lógica de negocio porque su rol principal debe limitarse a mostrar información y capturar la interacción del usuario.

Separar responsabilidades evita que la interfaz gráfica se convierta en una clase difícil de mantener, con código mezclado que combina eventos, validaciones y cálculos en un mismo lugar.

En el contexto de Java Swing, esta separación significa que los formularios, botones y cuadros de texto solo se encargan de recoger datos (entradas del usuario) y mostrar resultados (salidas), mientras que las validaciones, cálculos y reglas se trasladan al Controlador y al Modelo.

2.2 Código

2.2.1. Eventos

En lugar de colocar toda la lógica directamente dentro del evento `ActionListener`, es recomendable extraerla a un método privado dentro de la Vista, ya que de esta forma se evita duplicar código en caso de volver a necesitar la función.

```
private void intentarLogin() {...}  
btnIngresar.addActionListener(e -> intentarLogin());
```

2.2.2. Vista «tonta»

La Vista no debe leer atributos directamente, sino pedirlos a través de getters, de esta forma se implementa el principio de **encapsulamiento**, permitiendo validar o formatear valores sin romper la interfaz.

```
JLabel lblSaldo = new JLabel();  
private void refrescarSaldo() {  
    // Vista no sabe dónde se guarda realmente  
    int saldo = ruleta.getSaldo();  
    lblSaldo.setText("Saldo: $" + saldo);  
}
```

```
// Tras jugar o recargar:  
refrescarSaldo();
```

2.2.3. Uso de enum en las apuestas

Un **enum** en Java es un tipo de clase que permite definir un conjunto fijo de valores constantes con el objetivo de evitar el uso de literales sueltos (como caracteres o cadenas repetidas en el código). En este caso se define el enum **TipoApuesta** con los valores ROJO, NEGRO, PAR e IMPAR , que reemplaza caracteres como «R» o cadenas como «Par» incrustados y disperso en el código.

```
// Definición  
public enum TipoApuesta {  
    ROJO, NEGRO, PAR, IMPAR  
}  
  
// Uso en Ruleta  
public boolean evaluarResultado(int numero, TipoApuesta tipo) {  
    return switch (tipo) {  
        case ROJO -> esRojo(numero);  
        case NEGRO -> !esRojo(numero);  
        case PAR -> numero % 2 == 0;  
        case IMPAR -> numero % 2 != 0;  
    };  
}
```

En la vista, al leer el valor seleccionado en el JComboBox, se debe convertir al **enum**, por ejemplo:

```
TipoApuesta tipo = TipoApuesta.valueOf(  
    cboTipo.getSelectedItem().toString().toUpperCase()  
);
```

2.2.4. Sesión

La sesión debe existir como una única instancia que se crea en el **Launcher** y se inyecta en todas las vistas. Su rol es manejar el registro, login y el usuario actual.

```
SessionController session = new SessionController();  
new VentanaLogin(session).mostrarVentana();
```

Primero, en el launcher, se crea una sola instancia de SessionController que representa la sesión de todo el sistema. Luego esa misma sesión se pasa a la primera ventana (VentanaLogin).

Nota: No se deben crear nuevas sesiones en otras ventanas, siempre se debe reutilizar esta de manera centralizada.

Con esto la lógica esperada es que el usuario ingrese sus credenciales, la vista llame a `session.iniciarSesion(user, pass)` y según el resultado decida a dónde navegar.

```
// En VentanaLogin, tras login OK
new VentanaMenu(session).mostrar();
```

Este código se ejecuta solo cuando el login fue exitoso, abriendo la ventana del menú reutilizando la misma sesión ya creada en el Launcher.

Esqueleto simple

```
public class SessionController {
    private Usuario usuarioActual;

    public void registrarUsuario(String u, String p, String n) {
        if (u == null || u.isBlank() || p == null || p.isBlank() || n == null || n.
isBlank())
            throw new IllegalArgumentException("Datos requeridos");
        this.usuarioActual = new Usuario(u, p, n);
    }

    public boolean iniciarSesion(String u, String p) {
        if (usuarioActual == null) return false;
        return usuarioActual.validarCredenciales(u, p);
    }

    public boolean hayUsuario() {
        return usuarioActual != null;
    }

    public String getNombreUsuario() {
        return hayUsuario() ? usuarioActual.getNombre() : "";
    }
}
```

```
public Usuario getUsuarioActual() {  
    return usuarioActual;  
}  
  
public void cerrarSesion() {  
    usuarioActual = null;  
}  
}
```

2. Referencias

- [1] [Documentación] Apuntes Java - Jesús. <https://jessustm.github.io/ICC490-1-P00/>
- [2] [Documentación] Apuntes List - Jesús. <https://jessustm.github.io/ICC490-1-P00/1.Apuntes/11.List/>
- [3] [Tutorial] Vincular un proyecto Java en IntelliJ IDEA con GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-01%5D%20GitHub%20-%20IntelliJIDEA.pdf>
- [4] [Tutorial] Cómo usar Git y GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-02%5D%20Uso%20de%20GitHub.pdf>
- [5] [Tutorial] Cómo crear un menú en Java (ciclo do-while). <https://tutospoo.jimdofree.com/tutoriales-java/estructuras-repetitivas/ciclo-do-while-1-men%C3%BA/>